

WFLOW-05: PagerDuty & ServiceNow Integration

Series: WFLOW — Workflows and Alert Notifications | **Notebook:** 5 of 10 | **Created:** January 2026 | **Last Updated:** 10/02/2026

Incident Management Automation

Integrate Dynatrace workflows with enterprise incident management platforms. This notebook covers PagerDuty and ServiceNow integration patterns, bi-directional sync, and incident lifecycle management.

Table of Contents

1. [PagerDuty Setup](#)
 2. [PagerDuty Workflow Tasks](#)
 3. [ServiceNow Setup](#)
 4. [ServiceNow Workflow Tasks](#)
 5. [Bi-Directional Sync](#)
 6. [Incident Lifecycle Patterns](#)
 7. [Production Hardening](#)
-

Prerequisites

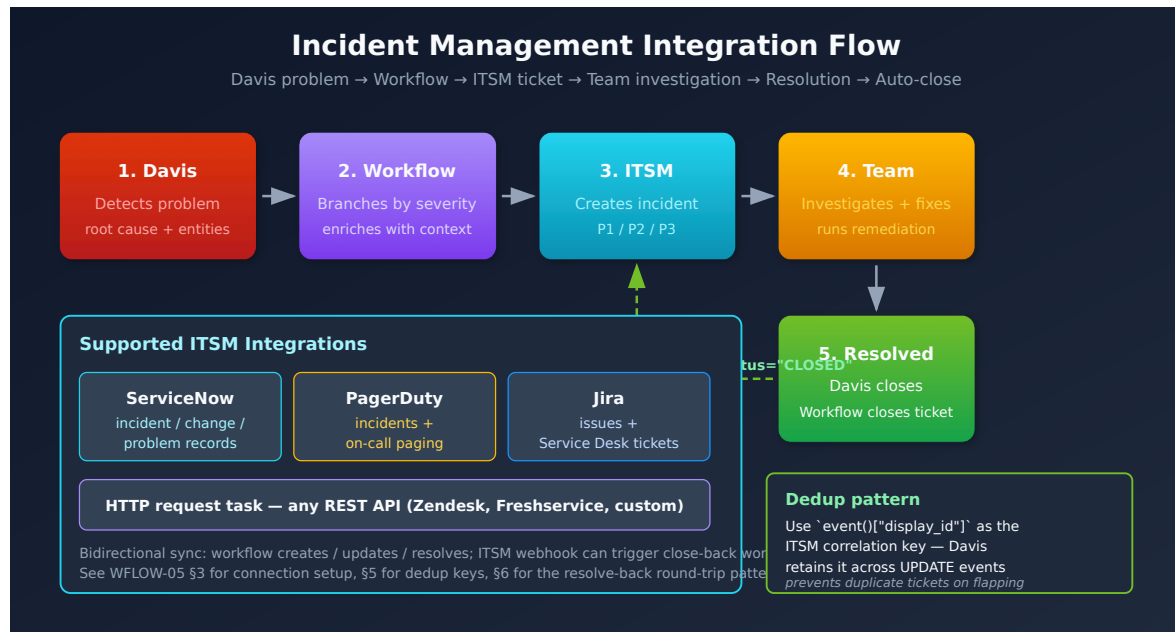
Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	<code>automation:workflows:write</code> , <code>automation:connections:write</code>
PagerDuty	Admin access to create services and integrations
ServiceNow	Admin access or integration user credentials

1. Integration Overview

Why Integrate?

Benefit	Description
Single pane of glass	Incidents tracked in one system
On-call management	Leverage PagerDuty/ServiceNow schedules
Audit trail	Complete incident history
Escalation	Built-in escalation policies
Metrics	MTTR, incident frequency reporting

Integration Flow



Why a Workflow (vs a Direct Webhook)

Davis can post to a webhook directly, but routing problems through a **workflow** buys four things a raw webhook cannot:

Capability	What the workflow adds
Payload shaping	Map Dynatrace fields to the exact incident schema the target expects, including required and custom fields
Retry + dead-letter	Re-attempt on transient failures; quarantine permanent failures instead of losing them (see §7)
Filtering	Decide which problems become incidents — drop informational events before they reach the ITSM queue
Link-back	Capture the returned incident number and write it back onto the Dynatrace problem for cross-reference

Reach for the native action first; drop to a raw HTTP action only when you need a field the action doesn't expose.

2. PagerDuty Setup

Creating PagerDuty Integration

1. In PagerDuty:

- Go to **Services** → Select or create service
- **Integrations** tab → **Add Integration**
- Select **Events API v2**
- Copy the **Integration Key** (routing key)

2. In Dynatrace:

- Go to **Settings** → **Integration** → **Connections**
- Click **+ Connection**
- Select **PagerDuty**
- Enter the Integration Key
- Name: `pagerduty-production`

Two connection types. The steps above create an **Events** connection (a routing key), which is what **Send event** uses: *"The Send event action uses the PagerDuty Events API v2 and requires a separate Events connection configured with a routing key."* **Create an incident** and the list actions use a different connection that holds a PagerDuty REST API key. Neither connection can stand in for the other.

Sources: [Set up PagerDuty Connector \(DT docs\)](#) — *"Use the PagerDuty Connector with a PagerDuty API key to automate incident creation and retrieve on-call information via the PagerDuty REST API."*

Multiple Services Pattern

Create separate connections for different routing:

Connection Name	PagerDuty Service	Use
<code>pagerduty-prod-critical</code>	Production-Critical	P1 incidents
<code>pagerduty-prod-standard</code>	Production-Standard	P2-P4 incidents
<code>pagerduty-platform</code>	Platform-Infrastructure	Infra issues

3. PagerDuty Workflow Tasks

PagerDuty has no separate resolve or acknowledge action. One action, **Send event** (`dynatrace.pagerduty:send-event`), covers the whole alert lifecycle: its **Event action** input is `trigger`, `acknowledge` or `resolve`. Send every event about one problem with the same deduplication key, through the same Events connection, so PagerDuty updates the existing alert instead of opening a new one.

Trigger an Alert

```
name: trigger_pagerduty_alert
action: dynatrace.pagerduty:send-event
input:
  connectionId: pagerduty-production
  eventAction: trigger
  severity: '{{ {1: "critical", 2: "error", 3: "warning", 4:
"info"}.get(event().get("event.severity") | int(5), "info") }}'
  summary: "[{{ event()['event.category'] }}] {{ event()['event.name'] }}"
  source: "dynatrace"
  component: "{{ event().get('root_cause_entity_id', 'unknown') }}"
  group: "{{ event().get('management_zones', ['default'])[0] }}"
  class: "{{ event()['event.category'] }}"
  customDetails: |
    # a JSON object, written as text
    {
      "problem_id": "{{ event()['display_id'] }}",
      "problem_url": "{{ problem_link() }}",
      "affected_entities": "{{ event()['affected_entity_ids'] | join(', ')
    }}",
      "root_cause": "{{ event().get('root_cause_entity_id', 'N/A') }}",
      "start_time": "{{ event()['event.start'] }}"
    }
  dedupKey: "dynatrace-{{ event()['display_id'] }}"
```

Resolve the Alert

Run when the problem closes:

```
name: resolve_pagerduty_alert
action: dynatrace.pagerduty:send-event
input:
  connectionId: pagerduty-production
  eventAction: resolve
  severity: info
  summary: "Resolved in Dynatrace: {{ event()['event.name'] }}"
  source: "dynatrace"
  dedupKey: "dynatrace-{{ event()['display_id'] }}"
```

Acknowledge the Alert

```
name: acknowledge_pagerduty_alert
action: dynatrace.pagerduty:send-event
input:
  connectionId: pagerduty-production
  eventAction: acknowledge
  severity: info
  summary: "Acknowledged from Dynatrace: {{ event()['event.name'] }}"
  source: "dynatrace"
  dedupKey: "dynatrace-{{ event()['display_id'] }}"
```

Dynatrace's own *Send problem as event to PagerDuty* template does the same in a single task: it sets the event action to `resolve` when the problem's status is `CLOSED`.

Opening an Incident Instead of an Alert

When you need a PagerDuty *incident* opened directly, with a service ID, priority or assignee, use **Create an incident** (`dynatrace.pagerduty:create-incident`) with the REST API connection. Its inputs differ from Send event (From, Title, Service ID, Priority ID, Urgency, Incident key), and it has no resolve counterpart in the connector.

Sources: [PagerDuty Connector actions \(DT docs\)](#) — "Trigger, acknowledge, or resolve an alert in PagerDuty using the Events API v2", "Creates an incident in your PagerDuty environment for a service."; [Send problem as event to PagerDuty template \(DT docs\)](#).

4. ServiceNow Setup

Creating ServiceNow Connection

1. In ServiceNow:

- Create integration user or use OAuth app
- Grant appropriate roles: `itil`, `rest_api_user`
- Note instance URL: `https://your-instance.service-now.com`

2. In Dynatrace:

- Go to **Settings** → **Integration** → **Connections**
- Click **+ Connection**
- Select **ServiceNow**
- Enter:
 - Instance URL
 - Basic-auth username/password **or** OAuth client ID/secret
 - Name: `servicenow-production`

Authentication Options

The ServiceNow connection supports two authentication methods — these are the only two the native connector accepts:

Method	Credentials	Best For
Basic Authentication	Username + password of a ServiceNow integration user	Quick setup, development
OAuth Client Credentials	Client ID + client secret	Production — avoids storing a user password

There is no API-key option for this connector. Store the chosen credential in the connection, never inline in a workflow task.

5. ServiceNow Workflow Tasks

The native ServiceNow connector exposes these operations, selected **by name** in the workflow builder. Create Incident is `dynatrace.servicenow:snow-create-incident` , and Comment on an incident is `dynatrace.servicenow:snow-comment-on-incident` , in Dynatrace's workflow samples. The other operations are shown by name only (`#operation:`): export a workflow that uses one to read its identifier and input names, rather than guessing them:

Operation	Use
Create Incident	Open an incident from a Davis problem
Resolve incident	Close the incident when the problem closes
Comment on an incident	Append work notes / status updates
Update record	Change fields on any table record (incidents included)
Search incidents	Look up an existing incident (e.g., by correlation ID)
Get Groups	Resolve assignment-group sys_ids
Create a vulnerability item	Open a VR item (security workflows)

Required fields for Create Incident: `Category` , `Subcategory` , `Impact` , `Urgency` , and `Assignment Group` are **required** — a create call missing any of them fails. `Correlation ID` is optional but strongly recommended: set it to the Dynatrace problem/event ID so later updates and resolves find the same incident instead of creating duplicates.

Create Incident

```
name: create_snow_incident
action: dynatrace.servicenow:snow-create-incident
input:
  connectionId: servicenow-production
  shortDescription: "[Dynatrace] {{ event()['event.name'] }}"
  description: |
    A problem has been detected by Davis.

    Problem Details:
    =====
    Problem ID: {{ event()['display_id'] }}
    Category: {{ event()['event.category'] }}
    Status: {{ event()['event.status'] }}
    Start Time: {{ event()['event.start'] }}

    Affected Entities:
    {{ event()['affected_entity_ids'] | join('\n') }}

    Root Cause Entity:
    {{ event().get('root_cause_entity_id', 'Pending analysis') }}

    View in Dynatrace:
    {{ problem_link() }}
  impact: '{{ {1: 1, 2: 1, 3: 2, 4: 3}.get(event().get("event.severity") |
int(5), 3) }}'
  urgency: '{{ {1: 1, 2: 1, 3: 2, 4: 3}.get(event().get("event.severity") |
int(5), 3) }}'
  category: "Software"
  subCategory: "Application"
  group: # the assignment group, picked in the editor
    id: "&sys_id of Platform Engineering&"
    displayName: "Platform Engineering"
  caller: "dynatrace.integration"
  correlationId: "DT-{{ event()['display_id'] }}"
```

Sources: [wftpl_sample_servicenow_incident_man.yaml \(Dynatrace GitHub\)](#) — "action: dynatrace.servicenow:snow-create-incident"; [wftpl_core_journey_auto-remediation.yaml \(Dynatrace GitHub\)](#) — "action: dynatrace.servicenow:snow-comment-on-incident".

Severity → Impact / Urgency. Dynatrace's unified `event.severity` is an integer **1–5** (1 = Critical ... 5 = Informational; see AIOPS-03), and it is what the Davis problem and `fetch`

events carry. ServiceNow Impact / Urgency run **1 (High) ... 3 (Low)**, so the Dynatrace scale compresses:

event.severity	ServiceNow Impact / Urgency
1 (Critical)	1 (High)
2	1 (High)
3	2 (Medium)
4	3 (Low)
5 (Informational)	skip — usually no incident

What event() actually carries. The problem record has event.severity on the 1–5 scale, not "CRITICAL" -style labels; on the validation tenant it arrives as a string ("3"), which is why the mappings above convert it with | int . It is experimental in the semantic dictionary, and from SaaS 1.348 it is no longer defaulted (WFLOW-04 §3), so int(5) sends an unset severity to the lowest tier. The same applies to the PagerDuty severity mapping in §3.

Comment on an Incident

```
name: comment_snow_incident
action: dynatrace.servicenow:snow-comment-on-incident
input:
  connectionId: servicenow-production
  # The incident number, not the correlation ID. Look it up first with a
  # Search incidents task, and check that task's result shape in a test run
  number: "INC0012345"
  comment: |
    [Automated Update from Dynatrace]
    Problem updated at {{ now() }}
    Current status: {{ event()['event.status'] }}
```

Resolve Incident

```
name: resolve_snow_incident
# Operation: "Resolve incident"
input:
  connection: servicenow-production
  correlation_id: "DT-{{ event()['display_id'] }}"
  close_code: "Resolved"
  close_notes: |
    Problem automatically resolved in Dynatrace.
    Resolution time: {{ event().get('event.end', now()) }}
```

6. Bi-Directional Sync

The native ServiceNow **workflow action is one-way** — it creates, comments on, resolves, and updates ServiceNow records, but ServiceNow-side changes do not flow back automatically. There are two ways to close the loop:

- **DIY polling (shown below)** — a scheduled workflow queries ServiceNow for Dynatrace-correlated incidents and writes their status back onto the Dynatrace problem. Portable and works today, but you own the polling cadence and rate budget.
- **Native bi-directional (ITOM)** — the ServiceNow-side Dynatrace app / ITOM Event Management ingests Dynatrace events and synchronises incident state both ways. This is the richest option for ITOM-centric shops; the **ALERT-04** notebook lays out the full maturity ladder and when to climb to it.

Sync Status from ServiceNow to Dynatrace (DIY polling)

Use a scheduled workflow to check incident status:

```

import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

const SNOW_URL = 'https://your-instance.service-now.com'; // add to Settings
> External requests

export default async function () {
  // A JavaScript task does not receive a connection object; keep the
  credential in the Credential Vault
  const cred = await credentialVaultClient.getCredentialsDetails({ id:
  'CREDENTIALS_VAULT-XXXXXXXXXXXX' });
  // Query ServiceNow for Dynatrace-related incidents
  const response = await fetch(
    `${SNOW_URL}/api/now/table/incident?` +
    `sysparm_query=correlation_id STARTSWITH DT-^state!=6`,
    {
      headers: {
        'Authorization': `Basic ${btoa(cred.username + ':' + cred.password)}`,
        'Accept': 'application/json'
      }
    }
  );

  const incidents = await response.json();

  // Process each incident
  for (const incident of incidents.result) {
    const problemId = incident.correlation_id.replace('DT-', '');
    // Add comment to Dynatrace problem with SNOW status
    // ...
  }

  return { processed: incidents.result.length };
}

```

Store Incident ID for Updates

Capture the ServiceNow incident number for later updates and for the link-back pattern (\$7):

```
tasks:
  - name: create_incident
    action: dynatrace.servicenow:snow-create-incident
    # ... configuration ...

  - name: store_incident_id
    action: dynatrace.automations:run-javascript
    predecessors: [create_incident]
    input:
      script: |
        import { result } from '@dynatrace-sdk/automation-utils';

        export default async function () {
          const incidentNumber = (await result('create_incident')).sys_id;
          // Store in workflow state or external system
          return { incident_number: incidentNumber };
        }
```

7. Incident Lifecycle Patterns

Complete Lifecycle Workflow

Handle open, update, and close events:

```
name: incident-lifecycle-management

trigger:
  type: davis-problem
  config:
    # Problem state: active or closed – fires on open and again on close
    entityTagsMatch: all
    entityTags:
      - key: env
        value: prod

conditions:
  - name: is_problem_open
    expression: '{{ event()["event.status"] == "ACTIVE" }}'
  - name: is_problem_closed
    expression: '{{ event()["event.status"] == "CLOSED" }}'

tasks:
  # CREATE: When problem opens
  - name: trigger_alert
    action: dynatrace.pagerduty:send-event
    conditions: [is_problem_open]
    input:
      connectionId: pagerduty-production
      eventAction: trigger
      source: dynatrace
      severity: critical
      summary: '{{ event()['event.name'] }}'
      dedupKey: "dynatrace-{{ event()['display_id'] }}"

  # RESOLVE: When problem closes
  - name: resolve_alert
    action: dynatrace.pagerduty:send-event
    conditions: [is_problem_closed]
    input:
      connectionId: pagerduty-production
      eventAction: resolve
      source: dynatrace
      severity: info
      summary: "Resolved in Dynatrace: {{ event()['event.name'] }}"
      dedupKey: "dynatrace-{{ event()['display_id'] }}"
```

Deduplication Strategy

Platform	Dedup Field	Pattern
PagerDuty	dedupKey	dynatrace-{problem_id}
ServiceNow	correlation_id	DT-{problem_id}

Same problem ID = same incident (no duplicates).

8. Production Hardening

An incident integration sits on the critical path: when it silently fails, problems open in Dynatrace but never reach the on-call queue. Harden it before you depend on it.

Failure Handling

Decide explicitly what happens on each failure mode rather than letting the workflow fail open:

Failure	Handling
ServiceNow 5xx (server error)	Retry with exponential backoff (e.g., 3 attempts)
ServiceNow 4xx (bad payload / auth)	Do not retry — write to a dead-letter store and alert the team; a 4xx won't fix itself on retry
Timeout (slow response)	Retry once; surface the failure on the Dynatrace problem as a comment
Rate limit reached	Log dropped events and continue; alert if the sustained drop rate crosses a threshold
DNS / connection failure	Retry; if persistent, alert via a secondary channel (e.g., a Slack action)
Auth token expired	Re-fetch the credential once per run from the connection/secret store; alert ops if it recurs

Dead-Letter Queue

Retries cover transient failures; **permanent** failures (4xx, schema mismatch) must not vanish. Write the failed payload to a durable store — a dedicated Grail bucket or an external queue — with a retention window (e.g., 30 days) so an operator can inspect and **replay** it after fixing the root cause. Without a dead-letter store, a malformed-payload bug means silently lost incidents with no audit trail.

Link Back to the Dynatrace Problem

Capture the ServiceNow incident number from the create response and write it onto the originating problem as a comment (the *Store Incident ID* pattern in §6). This gives responders a one-click path from the Dynatrace problem to the ServiceNow ticket — and is what lets later **Comment / Resolve** operations target the right incident via the correlation ID.

The integration is itself a production dependency, so monitor it like one — the queries below track execution volume, success rate, and failures so a broken connection surfaces as a metric, not as a silent gap in the incident queue.

Monitor Incident Integration

```
// Incident-management connector task executions
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
// and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
// any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
// `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
// `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
//   WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
//   send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
//   DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
//   is ERROR
//   .state.is_final                    true only on terminal records –
//   filter on it, otherwise a
//                                       single execution is counted once as
//   RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
//   task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
//   | limit 1
//   fetch dt.system.events, from:-7d
//   | filter event.kind == "WORKFLOW_EVENT"
//   | filter event.type == "ACTION_EXECUTION"
//   | filter dt.automation_engine.state.is_final == true
//   | filter contains(dt.automation_engine.action.app, "servicenow") or
//   contains(dt.automation_engine.action.app, "pagerduty") or
//   contains(dt.automation_engine.action.app, "jira")
//   | summarize executions = count(), by:{dt.automation_engine.action.app,
//   dt.automation_engine.action.function, dt.automation_engine.state}
//   | sort executions desc
```

```

// Failed incident management tasks
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records –
filter on it, otherwise a
//                                       single execution is counted once as
RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                       (was task.error) · duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| filter contains(dt.automation_engine.action.app, "servicenow") or
contains(dt.automation_engine.action.app, "pagerduty")
| fields timestamp, dt.automation_engine.action.function,
dt.automation_engine.state_info
| sort timestamp desc
| limit 25

```

```
// Connector activity per day
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records –
filter on it, otherwise a
//                                       single execution is counted once as
RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| filter contains(dt.automation_engine.action.app, "servicenow") or
contains(dt.automation_engine.action.app, "pagerduty")
| makeTimeseries executions = count(), interval:1d
```

Next Steps

With incident management configured, customize message templates:

Recommended Path

1. **WFLOW-06: Custom Notification Templates** - Rich formatting
2. **WFLOW-07: Problem-Triggered Remediation** - Auto-remediation
3. **WFLOW-08: JavaScript & HTTP Actions** - Custom integrations

Key Takeaways

- **PagerDuty: Send event** triggers, acknowledges and resolves through Events API v2 (routing key); **Create an incident** uses the REST API (API key)
 - **ServiceNow** uses a native connection (Basic or OAuth Client Credentials) with operations selected by name; `Category` , `Subcategory` , `Impact` , `Urgency` , `Assignment Group` are required on create
 - **Deduplication** via the correlation ID prevents duplicate incidents
 - **Lifecycle workflows** handle open/update/close events
 - **Bi-directional**: the native action is one-way; true state sync comes from the ServiceNow-side Dynatrace app / ITOM (see ALERT-04)
 - **Production hardening** — retry, dead-letter, link-back, and monitoring keep the integration trustworthy
-

Summary

In this notebook, you learned:

- Integration benefits and architecture
 - PagerDuty connection setup and tasks
 - ServiceNow connection setup and tasks
 - Bi-directional sync patterns
 - Incident lifecycle management
 - Deduplication strategies
 - Production hardening (failure handling, dead-letter, monitoring)
-

References

- [ServiceNow action \(DT docs\)](#)
 - [PagerDuty action \(DT docs\)](#)
 - [Send Dynatrace notifications to ServiceNow \(DT docs\)](#)
 - [Notification actions umbrella \(DT docs\)](#)
 - [HTTP request action \(DT docs\)](#)
 - [Davis Problems app \(DT docs\)](#)
 - [PagerDuty Events API v2 \(PagerDuty Developer\)](#)
 - [ServiceNow REST API reference \(ServiceNow Developer\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.